

Sprint 27 Structured Notes: Final Vanilla JavaScript Integration Project

0. What Sprint 27 Is About

Sprint 27 is about combining everything you have learned into one complete Vanilla JavaScript browser project.

Before Sprint 27, you learned many separate JavaScript skills:

- values
- variables
- strings
- numbers
- booleans
- conditions
- functions
- arrays
- loops
- objects
- callbacks
- higher-order array methods
- Sets and Maps
- DOM selection
- DOM updates
- events
- accessibility
- forms and validation
- dates and time
- CRUD
- localStorage and sessionStorage
- debugging
- regex
- classes
- algorithms
- data structures
- dynamic programming
- asynchronous JavaScript
- promises
- Fetch
- event loop basics
- modules

Sprint 27 changes the learning mode.

Earlier sprints asked:

Can I understand this one JavaScript concept?

Sprint 27 asks:

Can I combine many JavaScript concepts into one working app?

Main goal:

Build a small, maintainable, accessible, persistent, framework-free browser app with Vanilla JavaScript.

Simple meaning:

Sprint 27 teaches you how to use JavaScript concepts together instead of treating them as separate lessons.

By the end of Sprint 27, you should understand:

- how to plan a small JavaScript project
- how to organize files
- how to use JavaScript modules
- how to create a data model
- how to store app data in an array
- how to represent each item as an object
- how to create, read, update, and delete data
- how to validate a form
- how to use DOM selection
- how to create DOM elements safely
- how to use event listeners
- how to use event delegation
- how to save data to localStorage
- how to load data from localStorage
- how to use JSON.stringify()
- how to use JSON.parse()
- how to render data onto the page
- how to filter visible data
- how to sort visible data
- how to update accessible status messages
- how to use.textContent safely
- how to debug the app with console.table()
- how to follow a final QA checklist

Part A: Current Progress Before Sprint 27

1. What You Already Know

Sprint 27 assumes you already know the main pieces of beginner-to-intermediate Vanilla JavaScript.

You should not think of Sprint 27 as a completely new topic. It is an integration sprint.

Simple meaning:

Sprint 27 is where you connect previous skills into one complete project.

2. Core JavaScript Progress

Area	What You Learned	How It Helps in Sprint 27
Values	Store information like text, numbers, and booleans	App data is made from values
Variables	Use let and const to store values	DOM elements, task arrays, and settings are stored in variables
Strings	Work with text	Task titles, status messages, and storage keys are strings
Numbers	Calculate and compare values	Sorting, counts, and priority ranks may use numbers
Booleans	Represent true/false state	A task can be completed or not completed
Conditions	Choose what code runs	The app decides whether to add, delete, filter, or sort
Functions	Reuse logic	CRUD operations, rendering, and storage logic use functions
Arrays	Store ordered lists	All tasks are stored in an array
Objects	Store structured data	Each task is stored as an object
Loops	Repeat work	Rendering loops through all tasks
Array methods	map(), filter(), forEach(), sort()	The app updates, filters, renders, and sorts tasks
Debugging	Inspect problems	You use console.log(), console.table(), and DevTools

3. Browser JavaScript Progress

Area	What You Learned	How It Helps in Sprint 27
DOM selection	Find elements on the page	The app finds the form, inputs, buttons, list, filters, and status area
DOM updates	Change page content	The app displays tasks and status messages
DOM creation	Create new elements	The app creates list items, spans, and buttons for each task
Events	Respond to user actions	The app reacts to form submission, button clicks, and select changes
Forms	Collect user input	The app collects task title, due date, and priority
Validation	Check user input	The app blocks invalid task data
Accessibility	Keep UI state understandable	The app uses an aria-live status message
Storage	Save simple data	The app saves tasks in localStorage
Dates	Store time-related data	Tasks can have due dates and creation timestamps

4. Algorithm and Data Progress

Area	What You Learned	How It Helps in Sprint 27
Big O	Think about efficiency	You understand that looping through tasks grows with task count
Searching	Find matching data	Filtering is a kind of search-like operation
Sorting	Organize data	Tasks can be sorted by date or priority
Data structures	Choose containers	The app uses an array of objects as the main structure
Debugging process	Investigate calmly	You test each feature step by step

5. The Main Shift in Sprint 27

Before Sprint 27, you learned concepts separately.

Example:

```
const name = "Ada";
```

or:

```
button.addEventListener("click", () => {  
  console.log("Clicked");  
});
```

In Sprint 27, those ideas work together.

Example:

```
form.addEventListener("submit", event => {  
  event.preventDefault();  
  
  const title = titleInput.value.trim();  
  const task = createTask(title, dueDateInput.value, priorityInput.value);  
  
  tasks = addTask(tasks, task);  
  saveTasks(tasks);  
  renderTasks(tasks, taskList);  
});
```

This one flow uses:

- events
- forms
- strings
- objects
- arrays
- functions
- localStorage
- DOM rendering

Simple summary:

```
Sprint 27 turns many small JavaScript tools into one app-building workflow.
```

Part B: Sprint 27 Project Scope

6. The Project Type

The recommended Sprint 27 project is a small task app or study-planner app.

Example app idea:

Study Planner

The app lets the user:

- add a study task
- choose a due date
- choose a priority
- mark the task complete
- mark the task active again
- delete the task
- filter tasks
- sort tasks
- save tasks after refresh

7. Technologies Allowed

Use only:

HTML
CSS
Vanilla JavaScript
localStorage

Simple meaning:

Build the app with the browser tools you already learned.

8. Technologies Not Used

Do not use:

React
Vue
Angular
Svelte
Next.js
Vite
Webpack
Node backend
Database server

External UI library
Build tools

Why?

Because Sprint 27 is not about frameworks. It is about proving that you understand the core browser and JavaScript layer.

Beginner rule:

Learn to build the app with plain JavaScript before adding frameworks.

Part C: The Core App Mental Model

9. Basic Data Flow

The app follows this flow:

```
User enters task data
↓
JavaScript validates the form
↓
JavaScript creates a task object
↓
JavaScript adds the task object to an array
↓
JavaScript saves the array to localStorage
↓
JavaScript renders the array onto the page
```

This is the most important mental model in Sprint 27.

10. App State

App state means the current data the app is using.

In this project, the main state is:

```
let tasks = [];
```

Simple meaning:

```
tasks is the current list of all saved tasks.
```

When the user adds a task, the array changes.

When the user deletes a task, the array changes.

When the user marks a task complete, the array changes.

11. Rendered UI

Rendered UI means what the user sees on the page.

The UI should come from the data.

Simple rule:

```
Do not treat the page as the source of truth.  
Treat the tasks array as the source of truth.
```

The page should display whatever is currently inside the tasks array.

12. The Main Sprint 27 Pattern

Every time the data changes, do three things:

1. Update the tasks array.
2. Save the tasks array.
3. Render the page again.

Example:

```
tasks = deleteTask(tasks, taskId);  
saveTasks(tasks);  
updatePage("Task deleted.");
```

This pattern prevents two common bugs.

Bug 1:

The page changes, but the saved data does not change.

Bug 2:

The data changes, but the page does not update.

Part D: Main Project Features

13. Required Features

A strong Sprint 27 project should include these features:

- add a task
- display all tasks
- mark a task complete
- mark a completed task active again
- delete a task
- validate form input
- save tasks to localStorage
- load tasks from localStorage
- filter active tasks
- filter completed tasks
- show all tasks
- sort by due date
- sort by priority
- show accessible status messages
- use safe DOM updates
- use modules to organize code
- debug using `console.table()`

14. Optional Features

Optional improvements:

- edit task title
- clear completed tasks
- search tasks by title
- count active tasks
- count completed tasks
- detect overdue tasks
- use regex to validate a custom title pattern
- show empty-state text when no tasks exist

- add confirmation before deleting
- add better keyboard focus behavior

Do not add optional features until the required features work.

Beginner rule:

Finish the core app before adding extras.

Part E: Project File Structure

15. Why File Structure Matters

A project becomes harder to understand if all code is placed in one large file.

Sprint 27 introduces a simple project structure.

Recommended structure:

```
study-planner/  
  index.html  
  css/  
    styles.css  
  js/  
    main.js  
    storage.js  
    tasks.js  
    render.js
```

16. What Each File Does

File	Job
index.html	Page structure
css/styles.css	Page styling
js/main.js	Main app controller
js/storage.js	Save and load localStorage data
js/tasks.js	Create, update, delete, and transform task data

File	Job
js/render.js	Display tasks on the page

Simple meaning:

Each file should have one clear responsibility.

17. Why Splitting Files Helps

Splitting files helps because:

- code is easier to read
- bugs are easier to find
- each file has a clear purpose
- storage logic is separate from rendering logic
- task logic is separate from DOM logic
- the project feels closer to professional structure

Beginner rule:

Do not split files randomly. Split by responsibility.

Part F: JavaScript Modules

18. What Is a Module?

A module is a JavaScript file that can import or export code.

Simple meaning:

A module lets one JavaScript file share functions with another JavaScript file.

19. Loading a Module in HTML

Use this in `index.html`:

```
<script type="module" src="js/main.js"></script>
```

The important part is:

```
type="module"
```

This tells the browser:

```
Treat this JavaScript file as a module.
```

20. export

Use `export` when one file wants to share a function.

Example from `tasks.js`:

```
export function addTask(tasks, task) {  
  return [...tasks, task];  
}
```

Simple meaning:

```
Other files are allowed to import and use addTask.
```

21. import

Use `import` when one file wants to use something exported by another file.

Example from `main.js`:

```
import { addTask } from "../tasks.js";
```

Simple meaning:

```
Bring addTask from tasks.js into this file.
```

22. Module Path Rule

When importing your own files, include:

```
./
```

and usually include:

```
.js
```

Correct:

```
import { saveTasks } from "../storage.js";
```

Common beginner mistake:

```
import { saveTasks } from "storage";
```

Why this is wrong:

The browser does not know where storage is.

23. Modules and Script Timing

Module scripts are deferred by default.

Simple meaning:

The browser waits until the HTML is parsed before running the module script.

That reduces the chance of JavaScript trying to select elements before they exist.

Still, place the script near the end of the body for beginner clarity.

Part G: The Data Model

24. What Is a Data Model?

A data model is the shape of the data your app uses.

Simple meaning:

A data model says what information each task should have.

25. Task Object Shape

Each task can look like this:

```
const task = {
  id: "task-1",
  title: "Study Sprint 27",
  dueDate: "2026-05-10",
  priority: "high",
  completed: false,
  createdAt: "2026-05-07T10:30:00.000Z"
};
```

26. Task Properties

Property	Type	Meaning
id	string	Unique identifier for the task
title	string	The task text
dueDate	string	The date the task is due
priority	string	low, medium, or high
completed	boolean	Whether the task is finished
createdAt	string	When the task was created

27. Why Each Task Needs an id

A task title is not enough to identify a task.

Example:

```
const tasks = [
  { title: "Study JavaScript" },
  { title: "Study JavaScript" }
];
```

Two tasks can have the same title.

So the app needs a unique id.

Example:

```
const task = {  
  id: crypto.randomUUID(),  
  title: "Study JavaScript"  
};
```

Simple meaning:

The id lets the app update or delete the correct task.

28. crypto.randomUUID()

`crypto.randomUUID()` creates a unique string.

Example:

```
const id = crypto.randomUUID();  
console.log(id);
```

Possible output:

8b5c1f88-8e22-4d2e-91a7-4f59c5e901e2

Beginner meaning:

Use this when you need a unique id in a browser project.

29. createdAt

`createdAt` stores when the task was created.

Example:

```
const createdAt = new Date().toISOString();
```

Simple meaning:

Store the current time as a standard date string.

This can help with sorting later.

Part H: HTML Structure

30. Why HTML Comes First

Before JavaScript can select or update elements, the HTML must exist.

Simple app structure:

```
<main>
  <h1>Study Planner</h1>
  <form id="taskForm">...</form>
  <p id="statusMessage" aria-live="polite"></p>
  <ul id="taskList"></ul>
</main>
```

The HTML gives JavaScript targets to work with.

31. Full Beginner HTML Starter

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Study Planner</title>
    <link rel="stylesheet" href="css/styles.css">
  </head>
  <body>
    <main>
      <h1>Study Planner</h1>

      <form id="taskForm">
        <label for="titleInput">Task title</label>
        <input id="titleInput" type="text" required>

        <label for="dateInput">Due date</label>
        <input id="dateInput" type="date" required>
```



```

    <label for="priorityInput">Priority</label>
    <select id="priorityInput" required>
      <option value="">Choose priority</option>
      <option value="low">Low</option>
      <option value="medium">Medium</option>
      <option value="high">High</option>
    </select>

    <button type="submit">Add task</button>
  </form>

  <div>
    <label for="filterInput">Filter</label>
    <select id="filterInput">
      <option value="all">All</option>
      <option value="active">Active</option>
      <option value="completed">Completed</option>
    </select>

    <label for="sortInput">Sort</label>
    <select id="sortInput">
      <option value="created">Created order</option>
      <option value="date">Due date</option>
      <option value="priority">Priority</option>
    </select>
  </div>

  <p id="statusMessage" aria-live="polite"></p>

  <ul id="taskList"></ul>
</main>

<script type="module" src="js/main.js"></script>
</body>
</html>

```

32. Important HTML Parts

Form

```
<form id="taskForm">
```

The form groups the task inputs together.

Labels

```
<label for="titleInput">Task title</label>  
<input id="titleInput" type="text" required>
```

The `for` value matches the input `id`.

Simple meaning:

The label describes the input.

Required Fields

```
required
```

This tells the browser:

The user must fill this field before submitting.

Submit Button

```
<button type="submit">Add task</button>
```

This submits the form.

Status Message

```
<p id="statusMessage" aria-live="polite"></p>
```

This displays messages like:

Task added.
Task updated.
Task deleted.

The `aria-live="polite"` part helps assistive technology notice changes.

Task List

```
<ul id="taskList"></ul>
```

This is where JavaScript will render task items.

Part I: DOM Selection

33. What Is DOM Selection?

DOM selection means finding HTML elements with JavaScript.

Example:

```
const form = document.getElementById("taskForm");
```

Simple meaning:

Find the element with id="taskForm" and store it in the form variable.

34. DOM Elements Needed in Sprint 27

In `main.js`, select the main elements:

```
const form = document.getElementById("taskForm");
const titleInput = document.getElementById("titleInput");
const dateInput = document.getElementById("dateInput");
const priorityInput = document.getElementById("priorityInput");
const filterInput = document.getElementById("filterInput");
const sortInput = document.getElementById("sortInput");
const taskList = document.getElementById("taskList");
const statusMessage = document.getElementById("statusMessage");
```

35. Why Store DOM Elements in Variables?

This is easier:

```
statusMessage.textContent = "Task added.";
```

than repeatedly writing:

```
document.getElementById("statusMessage").textContent = "Task added.";
```

Beginner rule:

Select important elements once, store them in variables, then reuse those variables.

36. Null Selection Mistake

If JavaScript cannot find an element, the result is `null`.

Example:

```
const taskList = document.getElementById("wrongId");  
console.log(taskList); // null
```

Then this would fail:

```
taskList.textContent = "";
```

Why?

Because `null` does not have a `textContent` property.

Beginner debugging rule:

If DOM code fails, first check whether the selected element is null.

Part J: Forms and Validation

37. Why Forms Matter in Sprint 27

The form collects user input.

The task app needs three pieces of input:

- task title
- due date
- priority

38. Form Submission Event

Use the `submit` event:

```
form.addEventListener("submit", event => {  
  event.preventDefault();  
});
```

Simple meaning:

When the user submits the form, run this function.

39. preventDefault()

By default, a form may reload the page when submitted.

This line stops that default behavior:

```
event.preventDefault();
```

Simple meaning:

Do not reload the page. Let JavaScript handle the form.

40. checkValidity()

`checkValidity()` checks whether the form passes built-in HTML validation.

Example:

```
if (!form.checkValidity()) {  
  form.reportValidity();  
}
```

```
    return;  
}
```

Simple meaning:

If the form is invalid, show the browser's validation message and stop.

41. reportValidity()

`reportValidity()` tells the browser to show validation feedback.

Example:

```
form.reportValidity();
```

This can show messages like:

Please fill out this field.

42. trim()

Use `trim()` to remove extra spaces from the beginning and end of text.

Example:

```
const title = titleInput.value.trim();
```

Why?

Because this should not count as a useful task title:

" "

Beginner rule:

Trim user text before saving it.

43. Reading Form Values

Example:

```
const title = titleInput.value.trim();
const dueDate = dateInput.value;
const priority = priorityInput.value;
```

Simple meaning:

Get the current values from the form fields.

44. Resetting the Form

After adding a task, clear the form:

```
form.reset();
```

Simple meaning:

Put the form fields back to their initial empty state.

Part K: CRUD in Sprint 27

45. What Is CRUD?

CRUD means:

Letter	Meaning	Task App Example
C	Create	Add a new task
R	Read	Display tasks
U	Update	Mark a task complete or active
D	Delete	Remove a task

Simple meaning:

CRUD is the basic pattern for managing app data.

46. Create

Create means add a new task.

Example:

```
const newTask = createTask(title, dueDate, priority);
tasks = addTask(tasks, newTask);
```

47. Read

Read means display the current tasks.

Example:

```
renderTasks(tasks, taskList);
```

48. Update

Update means change an existing task.

Example:

```
tasks = toggleTask(tasks, taskId);
```

This changes `completed` from:

```
false
```

to:

```
true
```

or from:


```
true
```

to:

```
false
```

49. Delete

Delete means remove a task.

Example:

```
tasks = deleteTask(tasks, taskId);
```

Part L: Task Logic File

50. Purpose of tasks.js

The `tasks.js` file contains functions that work with task data.

It should not directly change the page.

It should not directly use localStorage.

Its job is only task data logic.

Simple meaning:

```
tasks.js manages the array and task objects.
```

51. createTask()

```
export function createTask(title, dueDate, priority) {  
  return {  
    id: crypto.randomUUID(),  
    title,  
    dueDate,  
    priority,  
  }  
}
```

```
    completed: false,  
    createdAt: new Date().toISOString()  
  };  
}
```

This function creates one task object.

Input:

```
title  
dueDate  
priority
```

Output:

```
a complete task object
```

52. Property Shorthand

This code:

```
return {  
  title,  
  dueDate,  
  priority  
};
```

is shorthand for:

```
return {  
  title: title,  
  dueDate: dueDate,  
  priority: priority  
};
```

Simple meaning:

```
If the property name and variable name are the same, JavaScript lets you write  
it once.
```

53. addTask()

```
export function addTask(tasks, task) {  
  return [...tasks, task];  
}
```

This returns a new array containing:

- all old tasks
- the new task at the end

Simple meaning:

Add the task without directly changing the original array.

54. Spread Syntax

This part:

```
...tasks
```

means:

Copy the existing task items into the new array.

Example:

```
const oldTasks = ["A", "B"];  
const newTasks = [...oldTasks, "C"];  
  
console.log(newTasks); // ["A", "B", "C"]
```

55. deleteTask()

```
export function deleteTask(tasks, taskId) {  
  return tasks.filter(task => task.id !== taskId);  
}
```

This keeps every task except the task with the matching id.

Simple meaning:

`filter()` creates a new array with only the tasks we want to keep.

56. toggleTask()

```
export function toggleTask(tasks, taskId) {  
  return tasks.map(task => {  
    if (task.id === taskId) {  
      return {  
        ...task,  
        completed: !task.completed  
      };  
    }  
  
    return task;  
  });  
}
```

This loops through all tasks.

If it finds the matching task, it returns a changed copy.

If it does not match, it returns the original task.

57. The ! Operator

This part:

```
!task.completed
```

means:

Use the opposite boolean value.

Examples:

```
!true // false  
!false // true
```

So this:

```
completed: !task.completed
```

means:

```
If completed is true, make it false.  
If completed is false, make it true.
```

Part M: Storage Logic

58. Why Storage Is Needed

Without storage, tasks disappear when the page reloads.

Example:

```
let tasks = [];
```

This array lives only in memory.

If the page refreshes, the script starts again, and the array becomes empty again.

localStorage solves this problem.

Simple meaning:

```
localStorage lets the browser remember simple data after refresh.
```

59. localStorage Stores Strings

localStorage can store string values.

Example:

```
localStorage.setItem("username", "Ada");
```

But tasks are an array of objects, not a simple string.

So the app must convert the tasks array to a string before saving.

60. JSON.stringify()

`JSON.stringify()` converts JavaScript data into JSON text.

Example:

```
const tasksText = JSON.stringify(tasks);
localStorage.setItem("studyPlannerTasks", tasksText);
```

Simple meaning:

Turn the tasks array into text so localStorage can save it.

61. JSON.parse()

`JSON.parse()` converts JSON text back into JavaScript data.

Example:

```
const savedText = localStorage.getItem("studyPlannerTasks");
const tasks = JSON.parse(savedText);
```

Simple meaning:

Turn saved text back into an array.

62. storage.js

```
const STORAGE_KEY = "studyPlannerTasks";

export function loadTasks() {
  try {
    const savedTasks = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    if (Array.isArray(savedTasks)) {
      return savedTasks;
    }
  }
}
```

```

    return [];
  } catch (error) {
    console.error("Could not load tasks:", error.message);
    return [];
  }
}

export function saveTasks(tasks) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}

```

63. STORAGE_KEY

```
const STORAGE_KEY = "studyPlannerTasks";
```

This avoids repeating the same storage key string many times.

Beginner rule:

Store repeated important strings in constants.

64. loadTasks()

`loadTasks()` tries to read saved tasks.

If there are saved tasks, it returns them.

If there are no saved tasks, it returns an empty array.

If the saved data is broken, it catches the error and returns an empty array.

65. try...catch

This part:

```

try {
  // risky code
} catch (error) {
  // fallback code
}

```

means:

Try to run the code. If it fails, handle the error safely.

This matters because `JSON.parse()` can fail if `localStorage` contains invalid JSON.

66. `Array.isArray()`

```
if (Array.isArray(savedTasks)) {  
  return savedTasks;  
}
```

This checks whether the loaded data is actually an array.

Simple meaning:

Do not trust saved data blindly.

67. `saveTasks()`

```
export function saveTasks(tasks) {  
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));  
}
```

This saves the current task array.

Simple meaning:

Convert the tasks array into text and store it in the browser.

Part N: Rendering Logic

68. What Is Rendering?

Rendering means displaying data on the page.

Simple meaning:

Take the tasks array and show it as HTML elements.

69. render.js

```
export function renderTasks(tasks, taskList) {
  taskList.textContent = "";

  tasks.forEach(task => {
    const item = document.createElement("li");

    const title = document.createElement("span");
    title.textContent = task.title;

    const dueDate = document.createElement("span");
    dueDate.textContent = `Due: ${task.dueDate}`;

    const priority = document.createElement("span");
    priority.textContent = `Priority: ${task.priority}`;

    const completeButton = document.createElement("button");
    completeButton.textContent = task.completed ? "Mark active" : "Mark
complete";
    completeButton.dataset.action = "toggle";
    completeButton.dataset.id = task.id;

    const deleteButton = document.createElement("button");
    deleteButton.textContent = "Delete";
    deleteButton.dataset.action = "delete";
    deleteButton.dataset.id = task.id;

    if (task.completed) {
      item.classList.add("completed");
    }

    item.append(title, dueDate, priority, completeButton, deleteButton);
    taskList.appendChild(item);
  });
}
```

70. Clearing the List First

```
taskList.textContent = "";
```

This removes the old rendered task list before rendering the new one.

Why?

If you do not clear the list first, duplicate items may appear.

Beginner rule:

```
When re-rendering a list, clear the old list first.
```

71. createElement()

```
const item = document.createElement("li");
```

This creates a new HTML element in memory.

Simple meaning:

```
Make a new list item using JavaScript.
```

It is not visible on the page until you append it.

72. textContent

```
title.textContent = task.title;
```

This safely puts text into the element.

Beginner rule:

```
Use textContent for user-controlled text.
```

Do not use this for task titles:

```
title.innerHTML = task.title;
```

Why?

Because `innerHTML` treats the string as HTML. User input should usually be treated as plain text.

73. dataset

```
completeButton.dataset.action = "toggle";  
completeButton.dataset.id = task.id;
```

This creates data attributes on the button.

Conceptual HTML result:

```
<button data-action="toggle" data-id="abc123">Mark complete</button>
```

Simple meaning:

Store useful information on the button so the click handler knows what to do.

74. Conditional Button Text

```
completeButton.textContent = task.completed ? "Mark active" : "Mark complete";
```

This uses a ternary operator.

Simple meaning:

If the task is completed, show "Mark active".
Otherwise, show "Mark complete".

Longer version:

```
if (task.completed) {  
    completeButton.textContent = "Mark active";  
} else {  
    completeButton.textContent = "Mark complete";  
}
```

75. `classList.add()`

```
if (task.completed) {  
  item.classList.add("completed");  
}
```

This adds a CSS class to completed tasks.

Simple meaning:

Completed tasks can look different visually.

Example CSS:

```
.completed {  
  text-decoration: line-through;  
}
```

76. `append()`

```
item.append(title, dueDate, priority, completeButton, deleteButton);
```

This places multiple child elements inside the list item.

Then:

```
taskList.appendChild(item);
```

puts the list item into the visible task list.

Part O: Main App Controller

77. Purpose of `main.js`

`main.js` connects everything.

It imports helper functions, selects DOM elements, stores app state, listens for events, saves data, and updates the page.

Simple meaning:

main.js is the control center of the app.

78. Importing Code

```
import { loadTasks, saveTasks } from "../storage.js";
import {
  createTask,
  addTask,
  deleteTask,
  toggleTask
} from "../tasks.js";
import { renderTasks } from "../render.js";
```

This brings functions from other files into `main.js`.

79. Loading Initial State

```
let tasks = loadTasks();
```

Simple meaning:

When the app starts, load saved tasks from localStorage.

If there are no saved tasks, `loadTasks()` returns an empty array.

80. updatePage()

```
function updatePage(message = "") {
  renderTasks(getVisibleTasks(), taskList);
  statusMessage.textContent = message;
  console.table(tasks);
}
```

This function does three useful things:

1. renders the visible tasks
2. updates the status message
3. prints the current task data as a table for debugging

Simple meaning:

Whenever the app changes, call `updatePage()`.

81. Default Parameter

```
function updatePage(message = "") {
```

This means:

If no message is provided, use an empty string.

Example:

```
updatePage("Task added.");  
updatePage();
```

Both calls work.

Part P: Filtering Tasks

82. What Is Filtering?

Filtering means showing only items that match a condition.

Examples:

```
Show all tasks.  
Show only active tasks.  
Show only completed tasks.
```

83. `getVisibleTasks()`

```
function getVisibleTasks() {  
  let visibleTasks = [...tasks];  
  
  if (filterInput.value === "active") {
```

```
    visibleTasks = visibleTasks.filter(task => !task.completed);
  }

  if (filterInput.value === "completed") {
    visibleTasks = visibleTasks.filter(task => task.completed);
  }

  return visibleTasks;
}
```

84. Why Copy the Array First?

```
let visibleTasks = [...tasks];
```

This creates a shallow copy.

Simple meaning:

Work with a copy so filtering and sorting do not accidentally damage the main tasks array.

85. Active Tasks

```
visibleTasks = visibleTasks.filter(task => !task.completed);
```

This keeps tasks where `completed` is false.

Simple meaning:

Show tasks that are not done yet.

86. Completed Tasks

```
visibleTasks = visibleTasks.filter(task => task.completed);
```

This keeps tasks where `completed` is true.

Simple meaning:

Show tasks that are already done.

Part Q: Sorting Tasks

87. What Is Sorting?

Sorting means putting items into a specific order.

Examples:

Sort by due date.
Sort by priority.
Sort by created order.

88. Sorting by Due Date

```
if (sortInput.value === "date") {  
    visibleTasks.sort((a, b) => a.dueDate.localeCompare(b.dueDate));  
}
```

This compares date strings.

If dates are stored as `YYYY-MM-DD`, string comparison works for simple sorting.

Example:

```
2026-05-01  
2026-05-10  
2026-06-01
```

These sort correctly as strings because the year comes first, then month, then day.

89. localeCompare()

```
a.dueDate.localeCompare(b.dueDate)
```

Simple meaning:

Compare two strings and decide which should come first.

90. Sorting by Priority

```
if (sortInput.value === "priority") {  
  const priorityRank = {  
    high: 1,  
    medium: 2,  
    low: 3  
  };  
  
  visibleTasks.sort((a, b) => {  
    return priorityRank[a.priority] - priorityRank[b.priority];  
  });  
}
```

91. Why Priority Needs a Rank

The words are:

```
high  
medium  
low
```

Alphabetical sorting would not give the order we want.

So we create a ranking object:

```
const priorityRank = {  
  high: 1,  
  medium: 2,  
  low: 3  
};
```

Simple meaning:

Convert priority words into numbers so they can be sorted correctly.

92. sort() Mutates Arrays

Important:

```
sort()
```

changes the array it is called on.

That is why we sort `visibleTasks`, not the original `tasks` array.

Beginner rule:

```
Copy before sorting if you do not want to mutate the original array.
```

Part R: Event Listeners

93. What Events Does the App Need?

The app needs to listen for:

Event	Element	Purpose
submit	form	Add a task
click	task list	Toggle or delete a task
change	filter select	Change visible tasks
change	sort select	Change task order

94. Submit Event

```
form.addEventListener("submit", event => {
  event.preventDefault();

  if (!form.checkValidity()) {
    form.reportValidity();
    return;
  }

  const title = titleInput.value.trim();
  const dueDate = dateInput.value;
```

```
const priority = priorityInput.value;

const newTask = createTask(title, dueDate, priority);

tasks = addTask(tasks, newTask);
saveTasks(tasks);

form.reset();
updatePage("Task added.");
});
```

Step-by-step:

1. The user submits the form.
2. JavaScript stops the page reload.
3. JavaScript checks whether the form is valid.
4. JavaScript reads the input values.
5. JavaScript creates a task object.
6. JavaScript adds it to the tasks array.
7. JavaScript saves the updated array.
8. JavaScript resets the form.
9. JavaScript renders the page again.
10. JavaScript shows "Task added."

95. Change Events

```
filterInput.addEventListener("change", () => {
  updatePage();
});

sortInput.addEventListener("change", () => {
  updatePage();
});
```

Simple meaning:

When the user changes the filter or sort option, re-render the page.

Part S: Event Delegation

96. The Problem Event Delegation Solves

Task buttons are created dynamically.

That means they do not exist when the page first loads.

Instead of adding a separate event listener to every button, add one listener to the parent list.

This is called event delegation.

Simple meaning:

Listen on the parent, then figure out which child button was clicked.

97. Task List Click Listener

```
taskList.addEventListener("click", event => {
  const button = event.target.closest("button");

  if (button === null) {
    return;
  }

  const action = button.dataset.action;
  const taskId = button.dataset.id;

  if (action === "toggle") {
    tasks = toggleTask(tasks, taskId);
    saveTasks(tasks);
    updatePage("Task updated.");
  }

  if (action === "delete") {
    tasks = deleteTask(tasks, taskId);
    saveTasks(tasks);
    updatePage("Task deleted.");
  }
});
```

98. event.target

`event.target` is the element that was actually clicked.

Example:

```
event.target
```

Simple meaning:

```
What did the user click?
```

99. closest()

```
const button = event.target.closest("button");
```

This finds the nearest button at or above the clicked element.

If the click was not inside a button, it returns `null`.

100. Guard Clause

```
if (button === null) {  
  return;  
}
```

This stops the function if the user did not click a button.

Simple meaning:

```
If this click is not useful, exit early.
```

101. Reading dataset Values

```
const action = button.dataset.action;  
const taskId = button.dataset.id;
```

This reads values from the clicked button.

Example button:

```
<button data-action="delete" data-id="abc123">Delete</button>
```

JavaScript reads:

```
action // "delete"  
taskId // "abc123"
```

102. Deciding the Action

```
if (action === "toggle") {  
  tasks = toggleTask(tasks, taskId);  
}  
  
if (action === "delete") {  
  tasks = deleteTask(tasks, taskId);  
}
```

Simple meaning:

```
Use the button's data-action value to decide what should happen.
```

Part T: Accessibility in Sprint 27

103. Why Accessibility Still Matters

The app is dynamic.

That means the page changes after user actions.

Examples:

- task added
- task deleted
- task marked complete
- list filtered
- list sorted

Some users may not notice changes unless the app communicates them properly.

104. aria-live

Use this HTML:

```
<p id="statusMessage" aria-live="polite"></p>
```

Then update it with JavaScript:

```
statusMessage.textContent = "Task added.";
```

Simple meaning:

When the message changes, assistive technology can announce the update.

105. Good Status Messages

Good examples:

```
Task added.  
Task updated.  
Task deleted.
```

Weak examples:

```
Done.  
OK.  
Changed.
```

Why weak?

Because they do not clearly explain what happened.

Beginner rule:

Status messages should be short but specific.

106. Use Native HTML First

Use real buttons:

```
<button>Delete</button>
```

Do not use random divs as buttons:

```
<div>Delete</div>
```

Why?

Because real buttons already support keyboard interaction and button semantics.

107. Safe Text Updates

Use:

```
element.textContent = userText;
```

Avoid:

```
element.innerHTML = userText;
```

Simple meaning:

```
User text should be treated as text, not HTML.
```

Part U: Debugging Sprint 27

108. Why Debugging Matters Here

Sprint 27 combines many concepts, so bugs can come from many places:

- DOM selection
- form input
- validation
- task creation

- array updates
- localStorage
- JSON parsing
- rendering
- event delegation
- filtering
- sorting

Debugging must be systematic.

109. Use console.table()

```
console.table(tasks);
```

This displays the task array as a table.

It is useful for arrays of objects.

Example task table:

id	title	dueDate	priority	completed
abc123	Study JS	2026-05-10	high	false

Simple meaning:

```
console.table() helps you inspect structured data clearly.
```

110. Good Debugging Questions

When the app breaks, ask:

1. Did JavaScript load?
2. Did DOM selection return the correct elements?
3. Is the form submit event running?
4. Is preventDefault() stopping reload?
5. Are input values correct?
6. Is the task object created correctly?
7. Does the tasks array update?
8. Does localStorage save the updated data?
9. Does localStorage load the correct data after refresh?
10. Does renderTasks() receive the expected array?
11. Are buttons getting the correct data-action and data-id?
12. Is event delegation reading the correct button?

13. Is filtering changing only the visible list?
14. Is sorting mutating the wrong array?
15. Is the status message updating?

111. Debugging localStorage

In DevTools, inspect localStorage.

Look for the key:

```
studyPlannerTasks
```

The value should look like JSON text.

Example:

```
[
  {
    "id": "abc123",
    "title": "Study Sprint 27",
    "dueDate": "2026-05-10",
    "priority": "high",
    "completed": false,
    "createdAt": "2026-05-07T10:30:00.000Z"
  }
]
```

112. Common Errors

Error: Cannot read properties of null

Possible cause:

```
JavaScript could not find an element.
```

Check:

- Is the id spelled correctly in HTML?
- Is the id spelled correctly in JavaScript?
- Is the script running after the HTML exists?

Error: JSON.parse fails

Possible cause:

```
localStorage contains broken JSON.
```

Fix:

- use try...catch
- clear localStorage during testing if needed

Error: Button click does nothing

Possible cause:

```
Event delegation is not finding the button or dataset values.
```

Check:

```
console.log(button);  
console.log(button.dataset.action);  
console.log(button.dataset.id);
```

Error: Deleted task comes back after refresh

Possible cause:

```
You updated the array but forgot to save it.
```

Correct pattern:

```
tasks = deleteTask(tasks, taskId);  
saveTasks(tasks);  
updatePage("Task deleted.");
```

Error: Task saves but does not appear

Possible cause:

You saved the data but forgot to render the page again.

Correct pattern:

```
saveTasks(tasks);  
updatePage("Task added.");
```

Part V: Full main.js Example

```
import { loadTasks, saveTasks } from "./storage.js";  
import {  
  createTask,  
  addTask,  
  deleteTask,  
  toggleTask  
} from "./tasks.js";  
import { renderTasks } from "./render.js";  
  
const form = document.getElementById("taskForm");  
const titleInput = document.getElementById("titleInput");  
const dateInput = document.getElementById("dateInput");  
const priorityInput = document.getElementById("priorityInput");  
const filterInput = document.getElementById("filterInput");  
const sortInput = document.getElementById("sortInput");  
const taskList = document.getElementById("taskList");  
const statusMessage = document.getElementById("statusMessage");  
  
let tasks = loadTasks();  
  
function getVisibleTasks() {  
  let visibleTasks = [...tasks];  
  
  if (filterInput.value === "active") {  
    visibleTasks = visibleTasks.filter(task => !task.completed);  
  }  
  
  if (filterInput.value === "completed") {  
    visibleTasks = visibleTasks.filter(task => task.completed);  
  }  
  
  if (sortInput.value === "date") {  
    visibleTasks.sort((a, b) => a.dueDate.localeCompare(b.dueDate));  
  }  
}
```

```

    }

    if (sortInput.value === "priority") {
      const priorityRank = {
        high: 1,
        medium: 2,
        low: 3
      };

      visibleTasks.sort((a, b) => {
        return priorityRank[a.priority] - priorityRank[b.priority];
      });
    }

    return visibleTasks;
  }

  function updatePage(message = "") {
    renderTasks(getVisibleTasks(), taskList);
    statusMessage.textContent = message;
    console.table(tasks);
  }

  form.addEventListener("submit", event => {
    event.preventDefault();

    if (!form.checkValidity()) {
      form.reportValidity();
      return;
    }

    const title = titleInput.value.trim();
    const dueDate = dateInput.value;
    const priority = priorityInput.value;

    const newTask = createTask(title, dueDate, priority);

    tasks = addTask(tasks, newTask);
    saveTasks(tasks);

    form.reset();
    updatePage("Task added.");
  });

  taskList.addEventListener("click", event => {
    const button = event.target.closest("button");

    if (button === null) {

```

```

    return;
  }

  const action = button.dataset.action;
  const taskId = button.dataset.id;

  if (action === "toggle") {
    tasks = toggleTask(tasks, taskId);
    saveTasks(tasks);
    updatePage("Task updated.");
  }

  if (action === "delete") {
    tasks = deleteTask(tasks, taskId);
    saveTasks(tasks);
    updatePage("Task deleted.");
  }
});

filterInput.addEventListener("change", () => {
  updatePage();
});

sortInput.addEventListener("change", () => {
  updatePage();
});

updatePage();

```

Part W: Full Concept Map

113. Concepts Connected in Sprint 27

Concept	Where It Appears
Variables	DOM elements, tasks array, temporary values
Strings	task title, due date, priority, status messages, storage key
Booleans	completed true/false
Objects	each task
Arrays	list of all tasks
Functions	createTask, addTask, deleteTask, toggleTask, saveTasks, loadTasks, renderTasks

Concept	Where It Appears
Modules	import/export between files
Form validation	required fields, checkValidity(), reportValidity()
Events	submit, click, change
DOM selection	getElementById()
DOM creation	createElement()
DOM updates	textContent, appendChild(), classList.add()
Event delegation	one click listener on taskList
localStorage	saving after refresh
JSON	converting tasks to/from strings
Dates	dueDate and createdAt
Accessibility	aria-live status message
Array methods	forEach(), filter(), map(), sort()
Debugging	console.table(), console.error()

114. Data Flow Concept Map

```

HTML form
  ↓
submit event
  ↓
read input values
  ↓
validate values
  ↓
create task object
  ↓
add task to tasks array
  ↓
save tasks to localStorage
  ↓
filter/sort visible tasks
  ↓
render task elements
  ↓
update status message

```

115. Button Click Concept Map

```
User clicks task button
↓
taskList click listener catches the event
↓
closest("button") finds the button
↓
dataset.action tells what to do
↓
dataset.id tells which task to change
↓
update tasks array
↓
save tasks
↓
render page again
```

Part X: Common Beginner Mistakes

116. Forgetting preventDefault()

Problem:

```
form.addEventListener("submit", event => {
  // missing preventDefault
});
```

Result:

The page may reload and lose temporary state.

Fix:

```
event.preventDefault();
```

117. Saving Objects Directly to localStorage

Problem:


```
localStorage.setItem("tasks", tasks);
```

This may store something like:

```
[object Object]
```

Fix:

```
localStorage.setItem("tasks", JSON.stringify(tasks));
```

118. Forgetting JSON.parse()

Problem:

```
const tasks = localStorage.getItem("tasks");
```

This returns a string, not an array.

Fix:

```
const tasks = JSON.parse(localStorage.getItem("tasks")) || [];
```

119. Using innerHTML with User Text

Problem:

```
title.innerHTML = task.title;
```

Fix:

```
title.textContent = task.title;
```

120. Forgetting to Save After Updating

Problem:

```
tasks = toggleTask(tasks, taskId);  
updatePage("Task updated.");
```

The page changes, but refresh may undo the update.

Fix:

```
tasks = toggleTask(tasks, taskId);  
saveTasks(tasks);  
updatePage("Task updated.");
```

121. Forgetting to Render After Saving

Problem:

```
tasks = addTask(tasks, newTask);  
saveTasks(tasks);
```

The data saves, but the page does not update immediately.

Fix:

```
tasks = addTask(tasks, newTask);  
saveTasks(tasks);  
updatePage("Task added.");
```

122. Mutating the Original Array During Sort

Problem:

```
tasks.sort((a, b) => a.dueDate.localeCompare(b.dueDate));
```

This changes the original task order.

Safer version:

```
const visibleTasks = [...tasks];  
visibleTasks.sort((a, b) => a.dueDate.localeCompare(b.dueDate));
```

123. Using Title Instead of id

Problem:

```
deleteTask(tasks, title);
```

Titles may repeat.

Fix:

```
deleteTask(tasks, taskId);
```

124. Not Checking Broken Stored Data

Problem:

```
const savedTasks = JSON.parse(localStorage.getItem(STORAGE_KEY));  
return savedTasks;
```

If storage is broken, the app can crash.

Safer version:

```
try {  
  const savedTasks = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];  
  
  if (Array.isArray(savedTasks)) {  
    return savedTasks;  
  }  
  
  return [];  
} catch (error) {  
  return [];  
}
```

Part Y: Final QA Checklist

125. Core Functionality Checklist

Before calling Sprint 27 complete, test these:

- Can I add a task?
- Can I reject an empty title?
- Can I reject a missing due date?
- Can I reject a missing priority?
- Can I mark a task complete?
- Can I mark it active again?
- Can I delete a task?
- Can I filter active tasks?
- Can I filter completed tasks?
- Can I show all tasks again?
- Can I sort by due date?
- Can I sort by priority?
- Does data survive page refresh?
- Does the app load safely when there are no saved tasks?
- Does broken localStorage fail safely?
- Are status messages updated?
- Am I using `textContent` for user text?
- Can I inspect tasks with `console.table()`?
- Can I explain the data flow out loud?

126. Code Organization Checklist

- Is storage logic in `storage.js`?
- Is task data logic in `tasks.js`?
- Is rendering logic in `render.js`?
- Is controller logic in `main.js`?
- Are imports and exports working?
- Does `index.html` load `main.js` as a module?
- Are file paths correct?

127. Accessibility Checklist

- Do inputs have labels?
- Are buttons real button elements?
- Does the status message use `aria-live="polite"`?
- Are status messages clear?

Can a user understand what changed?
Is user text inserted with `textContent`?

128. Debugging Checklist

Did I check the console for errors?
Did I inspect selected DOM elements?
Did I use `console.table(tasks)`?
Did I inspect `localStorage` in DevTools?
Did I test one feature at a time?
Did I verify that save and render both happen after data changes?

Part Z: Sprint 27 Summary

129. One-Sentence Summary

Sprint 27 teaches you to combine separate JavaScript concepts into one maintainable, accessible, persistent, framework-free browser app.

130. The Most Important Rule

Data changes must update the array, save storage, and re-render the page.

In code:

```
tasks = updateTasksSomehow(tasks);  
saveTasks(tasks);  
updatePage("Something changed.");
```

131. Final Beginner Mental Model

The `tasks` array is the source of truth.
`localStorage` remembers the `tasks` array.
`renderTasks()` shows the `tasks` array on the page.
event listeners change the `tasks` array after user actions.
`updatePage()` keeps the page synchronized with the data.

132. What You Should Be Able to Explain After Sprint 27

After completing Sprint 27, you should be able to explain:

- why the app uses an array of objects
- why each task needs an id
- why form submission uses `preventDefault()`
- why validation happens before task creation
- why `localStorage` needs `JSON.stringify()`
- why loading needs `JSON.parse()`
- why loading should use `try...catch`
- why rendering clears the list first
- why user text should use `textContent`
- why event delegation works for dynamic buttons
- why filtering uses `filter()`
- why updating uses `map()`
- why deleting uses `filter()`
- why sorting uses `sort()`
- why sorting should happen on a copy
- why status messages help accessibility
- why modules make the project easier to organize
- why every data change should save and render again

Final simple meaning:

Sprint 27 is your first complete Vanilla JavaScript app-building sprint.